

Lógica y funcionamiento del programa

Giselle Juliana Gómez, Joshua Castro Solarte

September 10, 2026

La idea para desarrollar el programa de Sweet Candy es ir construyéndolo por etapas, empezando desde la parte más básica y fundamental, como la manipulación de bits y la memoria, hasta llegar a la lógica del juego y la interfaz.

Lógica matemática y orden de creación de funciones

Se plantea desarrollar el programa por fases, empezando por las funciones que son necesarias para que todo lo demás pueda funcionar y, posteriormente, agregando las funciones relacionadas con la lógica del juego y la parte visual.

1. Etapa 1: Matemática de bits

Primero vamos a implementar las funciones necesarias para la manipulación de bits. Esta parte es fundamental porque las fichas deben estar empaquetadas y guardadas de forma lineal dentro de los bytes.

La idea es poder calcular dónde se encuentra cada ficha dentro de la memoria, teniendo en cuenta su posición, el bit inicial, el byte en el que se encuentra y el desplazamiento (*offset*).

Pensamos inicialmente en funciones como: Calcular índice, calcular bit inicial, calcular byte, calcular offset, obtener ficha, y escribir ficha. Las funciones principales que pensamos utilizar son:

```
unsigned char obtenerFicha(  
    const unsigned char* tablero,  
    int posicion  
);  
  
void escribirFicha(  
    unsigned char* tablero,  
    int posicion,  
    unsigned char ficha  
);
```

Estas funciones serían la conexión entre la lógica del juego y la forma en que las fichas realmente están guardadas en la memoria.

El tablero se almacenará como una secuencia continua de bits, utilizando exactamente 3 bits para representar cada ficha. La información comenzará en el primer byte del arreglo dinámico y, dentro de cada byte, los bits válidos se ocuparán desde el bit menos significativo hacia el más significativo, es decir, visualmente de derecha a izquierda, luego en el siguiente byte los bits sobrantes se leerán como si estuvieran detrás también yendo de derecha a izquierda, y así sucesivamente. Los bytes se tratarán como partes consecutivas de una misma secuencia: después del bit 7 de un byte, la trama continuará en el bit 0 del byte siguiente. Por esta razón, una ficha puede quedar completamente almacenada dentro de un byte o repartida entre dos bytes consecutivos; por ejemplo, una ficha que comienza en el bit 6 utilizará los bits 6 y 7 del byte actual y continuará con el bit 0 del siguiente byte. Para localizar cualquier ficha, su posición (fila, columna) se convertirá primero en una posición lineal mediante $\text{índice} = \text{fila} \times \text{columnas} + \text{columna}$, y posteriormente se calculará $\text{bitInicial} = \text{índice} \times 3$. A partir de este valor se obtendrá el byte donde comienza mediante $\text{bitInicial} / 8$ y el offset mediante $\text{bitInicial} \% 8$; este último representa cuántas posiciones hay que desplazarse desde el

bit menos significativo del byte para encontrar el inicio de la ficha. La lectura y modificación de las fichas se realizará directamente sobre estos bits mediante máscaras y desplazamientos bitwise, contemplando cuando los 3 bits se encuentren repartidos entre dos bytes. Finalmente, para eliminar una ficha, sus bits no serán retirados físicamente de la secuencia, sino reemplazados por uno de los códigos de 3 bits reservados para representar una posición vacía; posteriormente, el proceso de reorganización desplazará las fichas necesarias y sobrescribirá las posiciones correspondientes, conservando siempre la representación compacta del tablero.

2. Etapa 2: Memoria

Después de tener clara la parte de los bits, debemos implementar lo relacionado con el manejo dinámico de la memoria del tablero.

En esta etapa se busca poder reservar, liberar, redimensionar y copiar la memoria cuando sea necesario.

Algunas de las variables que pensamos manejar son:

`unsigned char* tablero;` `int filas;` `int columnas;` `int bytesUsados;` `int bytesReservados.`

La idea es que la memoria utilizada corresponda realmente al tamaño actual del tablero y que, cuando cambien sus dimensiones, podamos reorganizar la memoria cuando sea necesario.

3. Etapa 3: Tablero

Cuando ya tengamos definida la forma en que vamos a manejar la memoria, podemos empezar a trabajar directamente con el tablero.

En esta etapa se desarrollarían funcionalidades como: Crear el tablero, generar las fichas, mostrar las fichas, mostrar binario y eliminar fichas.

También vamos a implementar la función de gravedad. Se plantea hacerla sin re-dimensionar la memoria, simplemente compactando las fichas de cada columna hacia abajo.

4. Etapa 4: Combinaciones

En esta sección planeamos desarrollar la funcionalidad para detectar las combinaciones de fichas y así poder eliminarlas, agregándole la verificación de una misma ficha contenida en una fila y en una columna y que no afecte la eliminación general. No vamos a detectar horizontal ni verticalmente al mismo tiempo.

Pensamos manejar esta parte con funciones como:

`detectarHorizontales();` `detectarVerticales();` `verificacionCruces()`

5. Etapa 5: Cascadas

Esta es probablemente una de las partes que más dependiente de las anteriores, porque para hacer una cascada necesitamos tener funcionando la gravedad, la generación de fichas, la detección de combinaciones y la eliminación.

El proceso que pensamos seguir sería:

Gravedad → Generar fichas → Buscar combinaciones → ¿Hay combinaciones? → Eliminar → Actualizar puntos

Si después de generar nuevas fichas aparecen nuevas combinaciones, se vuelve a repetir el proceso. El bucle continúa hasta que ya no haya más combinaciones, y de esta manera se completa la cascada.

6. Etapa 6: Eliminación e inserción

En esta etapa se implementaría la posibilidad de agregar o eliminar tanto filas como columnas del tablero, teniendo en cuenta la restricción del 65%

`agregarFila();` `eliminarFila();` `agregarColumna();` `eliminarColumna()`

Estas operaciones deben poder realizarse en cualquier posición válida del tablero, incluyendo posiciones intermedias.

7. Etapa 7: Partida e interfaz

Finalmente se implementaría la parte relacionada con la partida y la interacción con el usuario. Aquí se manejarían elementos como: Turnos, puntuación, estadísticas, menú del usuario, impresión del tablero y visualización en formato binario.

También se guardarían las acciones y resultados de la partida mediante variables como: `eliminacionesUsuario`; `fichasEliminadas`; `combinaciones`; `cascadasActuales`; `puntuacion`.

La idea es que esta parte sea manejada desde el `main`.

Organización del programa

La idea es separar las diferentes funcionalidades del programa en archivos `.h` y `.cpp`, para que cada parte tenga una función específica y el `main` se encargue principalmente de coordinar el funcionamiento general.

Se divide en los siguientes módulos, cada uno con su correspondiente archivo `.h` y `.cpp`: `bits`, `memoria`, `tablero`, `combinaciones`, `reorganizacion`, `estructura`, `juego` e `interfaz`. El único archivo principal es `main.cpp`, desde el cual se ejecuta el programa.

Flujo del funcionamiento del programa

A continuación mostramos el diagrama de flujo que seguiría el programa desde el momento en que inicia, llamando las funciones necesarias para que el juego funcione.

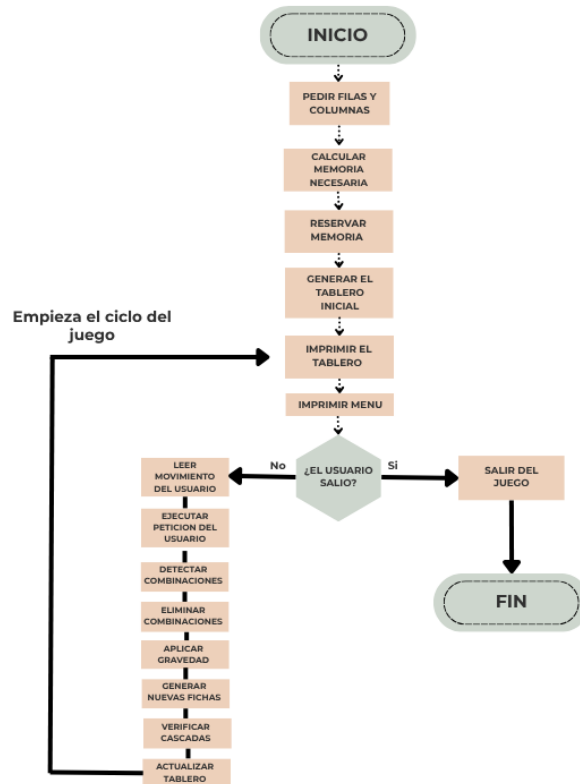


Figure 1: Diagrama de flujo del juego.